

FOR SPRING DEVELOPERS MOVING TO A NODE CODEBASE

Everything That Transfers

Spring Boot to NestJS, the flagship

Most of what you know transfers exactly. Nine things do not, and those are the expensive ones.

One payment system, built twice, on one docker compose file. Same contract, same request, separate databases, because one table with two object relational mappers writing to it is a defect generator rather than a comparison.

The sheet is the transfer ledger from the end of the video, with the code that each row depends on. The repository is the full runnable truth; this is the one page you pin next to the editor.

Measured on Java 25, Spring Boot 4.1.1, Node 24.21.0, NestJS 12.0.3, TypeScript 6.0.3, TypeORM 1.1.1 and PostgreSQL 18, on 19 September 2026. Those came off the lock file and the Dockerfiles rather than the package.json ranges, which is the difference between what is declared and what actually installs.

Every version on this sheet was current on 19 September 2026 and will not stay current. The method is the durable part: run the command, read what came back, and write down the number rather than the expectation.

In scope

- What transfers exactly, and it is most of it
- What transfers with different wiring
- The nine things that do not transfer at all
- Validation that actually runs, and why the annotation alone does not
- Entities: the decorator resembles the annotation, the lifecycle does not
- Why a race exists in a single threaded runtime

Out of scope

- Which framework is better. Neither, and the question wastes a migration
- Java the language against TypeScript the language, which is a different video
- Microservices, messaging and deployment, which are their own episodes
- Benchmarks. The numbers here measure specific behaviours, not throughput
- Anything about how this channel is produced, which is not the subject



- Graceful shutdown, and why it takes two separate fixes

Every figure on this page was measured rather than estimated, including the ones that contradicted the plan: the opening race was drafted at seven duplicate payments and the machine produced fifty.

01 The transfer ledger, in one table

Controllers, services, constructor injection	transfers exactly
DTO separate from domain, migrations in version control	transfers exactly
Invariants in the database, validation annotations	transfers exactly
Component scanning	becomes an explicit module declaration
@Valid on a parameter	becomes one global pipe
A derived query method	becomes a generic repository you write against
Futures	become promises
Interfaces at runtime, nominal typing, readonly	does not transfer
A managed entity, @Transactional semantics, ThreadLocal context	does not transfer
Checked exceptions, an executable jar	does not transfer

The third column is the one that costs money. Everything in it is a thing a Spring developer reasonably expects to be true and is not.

03 The annotation is not the enforcement

A TypeScript field declared as a number is not a number at runtime. The annotation is erased by the compiler, so nothing checks what arrives on the wire. Send an empty body to an endpoint with three required fields and you get 201 Created and a payment with no amount.

Spring appears to do better, and the reason is worth knowing precisely: it is Jackson binding the request before your code runs, not the type system. Send a quoted string where a number belongs and Spring hands your service a Long. Nest hands your service the string, still quoted, now pretending to be a number.

The fix is two things and you need both: decorators that describe the rules, and one pipe that runs them. Without transform the decorators have nothing to run against, because the object is still a plain parsed JSON object rather than an instance of the class.

```
nestjs-api/src/payments/dto/create-payment.dto.ts

import { IsIn, IsInt, IsNotEmpty, IsString, Min } from 'class-validator';

export class CreatePaymentRequest {
  @IsInt({ message: 'amountInMinorUnits must be an integer number of minor units' })
  @Min(1, { message: 'amountInMinorUnits must be at least 1' })
  amountInMinorUnits!: number;

  @IsIn(['USD', 'EUR', 'GBP', 'ZAR'], { message: 'currency must be a supported ISO code' })
  currency!: string;

  @IsString()
  @IsNotEmpty({ message: 'idempotencyKey is required' })
  idempotencyKey!: string;
}
```

```
nestjs-api/src/main.ts

app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,           // strip properties the class did not declare
    forbidNonWhitelisted: true, // and reject the request if one appears
    transform: true,          // make it a real instance, so the decorators run
  }),
);
```

transform is the flag people leave off. Without it the decorators are attached to a class nothing ever constructs.

An empty request body returned 201 Created on both stacks before validation was added.

Run on a real machine, 2026-09-19. Reproduce it with the scripts in the course repository.

A quoted string amount arrived as a Long in Spring and as a string in Nest, from the identical request.

Run on a real machine, 2026-09-19. Reproduce it with the scripts in the course repository.



whitelist strips undeclared properties, forbidNonWhitelisted rejects the request, transform produces a class instance.

`docs.nestjs.com`, checked 2026-09-19

04 The decorator resembles the annotation. The lifecycle does not.

Line for line, a TypeORM entity looks like a JPA entity, and that resemblance is the trap. It holds right up to the moment you rely on dirty checking.

A JPA entity inside a transaction is MANAGED: change a field and Hibernate writes it back for you at flush time. A TypeORM entity is a plain object. Change a field and precisely nothing happens until you call save.

The other surprise is the bigint. A column declared bigint comes back as a STRING, because the driver will not silently lose precision above two to the fifty third. Declare the field as a string and convert at the boundary, or discover it in production when an amount is concatenated instead of added.

```
nestjs-api/src/payments/payment.entity.ts

@Entity('payments')
@Unique('uq_payments_idempotency_key', ['idempotencyKey'])
export class PaymentEntity {
  @PrimaryGeneratedColumn('uuid')
  id!: string;

  @Column({ type: 'bigint' })
  amountInMinorUnits!: string; // a string, deliberately

  @Column({ type: 'varchar', length: 3 })
  currency!: string;

  @Column({ type: 'varchar', length: 100, name: 'idempotency_key' })
  idempotencyKey!: string;

  @CreateDateColumn({ name: 'created_at' })
  createdAt!: Date;
}
```

A bigint column round-trips through the driver as a string, to avoid losing precision above 2^{53} .

Copied from the course repository, github.com/code-with-sam-dev/spring-to-node

Changing a field on a TypeORM entity writes nothing until `save()` is called. There is no dirty checking.

docs.nestjs.com, checked 2026-09-19

Two mappers on one table is a defect generator: Hibernate wrote `amount_in_minor_units` and TypeORM wrote `amountInMinorUnits` into the same table, and the second service to start crashed on a not-null violation. Separate databases.

Run on a real machine, 2026-09-19. Reproduce it with the scripts in the course repository.

05 Single threaded does not mean uninterrupted

Fifty concurrent retries of one idempotency key produced fifty payments. The instinct is to say JavaScript is single threaded so there cannot be a race, and that instinct is exactly backwards.

Every await is a point at which the runtime is free to start work on another request. Two requests both look up the key. Both awaits yield. Both find nothing. Both insert. The thread never ran two things at once and you still have two rows.

So the lookup is an optimisation for the common retry, and the unique constraint is the guarantee. Try the insert and let Postgres be the one that refuses. Losing that race is the NORMAL outcome, not an error: the other request already created the payment, so return it.

```
nestjs-api/src/payments/payments.service.ts

// The lookup is an optimisation. The constraint is the guarantee.
const existing = await this.payments.findOne({
  where: { idempotencyKey: request.idempotencyKey },
});
if (existing) return toPayment(existing);

try {
  return toPayment(await this.payments.save(entity));
} catch (error) {
  // 23505 is Postgres for unique_violation. Losing this race is normal.
  if (error instanceof QueryFailedError && error.driverError?.code === '23505') {
    const winner = await this.payments.findOne({
      where: { idempotencyKey: request.idempotencyKey },
    });
    if (winner) return toPayment(winner);
    throw new ConflictException('Duplicate idempotency key');
  }
  throw error;
}
```

And the constraint that makes it true, which will not deploy onto dirty data

```
ALTER TABLE payments
  ADD CONSTRAINT uq_payments_idempotency_key UNIQUE (idempotency_key);
-- ERROR: could not create unique index "uq_payments_idempotency_key"
-- DETAIL: Key (idempotency_key)=(race-1789810120168) is duplicated.
```

Every tutorial adds a unique constraint to an empty table. Adding one to a table that already violates it is the real sequence, and it is the argument for putting the constraint in on day one rather than the day you need it.

Fifty concurrent requests with one idempotency key created fifty payments in 406ms before the constraint existed, and one after it.

Run on a real machine, 2026-09-19. Reproduce it with the scripts in the course repository.



The constraint refused to deploy until 49 duplicate rows were deleted.

Run on a real machine, 2026-09-19. Reproduce it with the scripts in the course repository.

23505 is the Postgres error code for unique_violation.

PostgreSQL 18 error codes, checked 2026-09-19

06 Five defects against five layers of tests

Every combination was run rather than argued about. A pure domain test caught nothing, because it never touches the framework. A wiring test caught nothing, and would have caught the missing provider if it imported the real module instead of building its own. A mocked contract test caught validation only.

Three of the five defects were invisible to every layer. A wrong column name, a removed unique constraint and a removed idempotency lookup all passed everything, because constraints and migrations are not testable properties of your CODE. They are properties of the DATABASE.

And a concurrency test needs enough concurrency. At ten requests the suite went green while the invariant was already broken. At fifty it failed.

Three of five injected defects passed all five test layers.

Run on a real machine, 2026-09-19. Reproduce it with the scripts in the course repository.

The concurrency test passed at 10 concurrent requests and failed at 50, against the same broken code.

Run on a real machine, 2026-09-19. Reproduce it with the scripts in the course repository.

Know what each layer can see, and put a guard where a test cannot reach.

A judgement, not a documented fact.

07 A container that takes ten seconds to die, and the two fixes it needs

Send the container a termination signal and the uptime goes UP. That is not a framework bug, it is three correct things meeting.

The image runs node directly, so node is process one. On Linux, process one gets no default signal handling: a signal with no explicit handler is ignored rather than acted on. And Node installs no default handler for termination, because normally it is not process one. So the signal lands nowhere, the grace period runs out, and the process is killed mid request.

Two fixes, and you need both. One decides whether the signal arrives. The other decides what happens when it does.

```
compose.yaml, and nestjs-api/src/main.ts
```

```
services:
  nestjs-api:
    init: true           # a real init at PID 1, so the signal is delivered
```

```
// and in the application
app.enableShutdownHooks(); // so something meaningful happens when it arrives
```

Either one alone leaves the container dying by force. With both, the in-flight request completes on both stacks.

Without init, the container reported "Up 16 minutes" before SIGTERM and "Up 17 minutes" after it.

Run on a real machine, 2026-09-19. Reproduce it with the scripts in the course repository.

Linux does not apply default signal dispositions to PID 1: a signal with no handler is ignored, not defaulted.

A judgement, not a documented fact.

With both fixes, the in-flight request completed on both stacks.

Run on a real machine, 2026-09-19. Reproduce it with the scripts in the course repository.



Before you call a NestJS service production ready

1. A global validation pipe with whitelist, forbidNonWhitelisted and transform
2. Every request object is a class with decorators, not an interface
3. A bigint column is read as a string and converted at the boundary
4. Every invariant you actually care about has a database constraint behind it
5. You have run the concurrency test at a concurrency that can fail
6. The idempotency path handles losing the race as a normal outcome
7. init: true in compose, and enableShutdownHooks in the application
8. Migrations are in version control and synchronize is off
9. A fresh clone runs docker compose up and both services answer



Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

Facebook

<https://www.facebook.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

☕ Buy me a coffee

<https://buymeacoffee.com/codewithsam>

Buy me sadza

<https://buymeacoffee.com/codewithsam>

The course repository

<https://github.com/code-with-sam-dev/spring-to-node>

NestJS docs

<https://docs.nestjs.com>

TypeORM docs

<https://typeorm.io>

Spring Boot reference

<https://docs.spring.io/spring-boot/index.html>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Node.js is a trademark of the OpenJS Foundation. Java is a trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. PostgreSQL is a trademark of the PostgreSQL Community Association. NestJS, TypeORM and TypeScript are the property of their respective owners. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-09-19 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.